Ex. No. 6	JavaScript Functions: Task Tracker
Date of Exercise	19.10.2026
GitHub Host Link	https://git.karunya.edu/pages/URK24CS1040/web-tech-lab/

Aim

To design and develop a Task Tracker web application using JavaScript functions, in which tasks can be added, completed, deleted and filtered, with the page structure written in HTML, the styling in CSS and every feature implemented as a small, reusable function in a separate JavaScript file.

Description

Functions: A function is a named block of code that performs one job and can be called whenever needed. Each feature of the application is a small function (createTask, addTask, toggleTask, deleteTask, getVisible, renderTasks), and the page simply calls them. This keeps every function to one job, allows reuse, and means a bug fixed in one function is fixed for every caller.

Parameters and return values: A function receives input through parameters and hands a result back with return. createTask(text) takes the task text and returns a new task object { id, text, done }.

State: The application keeps its data in one place: a tasks array and a filter variable. Functions change the state, and renderTasks() redraws the screen from it.

Array methods and callbacks: Functions can be passed to other functions as callbacks. filter() keeps the items that pass a test (deleteTask, getVisible, countOpen), map() transforms every item (toggleTask) and forEach() runs a function for each item (renderTasks). Arrow functions such as t => !t.done are used as short callbacks.

DOM manipulation: document.getElementById() and querySelectorAll() select the page elements. createElement(), appendChild() and textContent build each list row (taskItem), and classList.toggle() highlights the active filter button.

Event handling: addEventListener() connects the page to the functions. handleSubmit is passed to the form's submit event without brackets so that it runs only when the event fires; event.preventDefault() stops the page from reloading. Click and change events are used for the Delete button, the checkbox and the filter buttons.

Web storage: localStorage.setItem() with JSON.stringify() saves the tasks array in the browser, and JSON.parse() with localStorage.getItem() loads it again, so the tasks remain after the page is reloaded. saveAndRender() is the shared helper that saves and redraws after every change.

HTML and CSS: The HTML holds empty placeholders (form, ul and p elements with ids) that the script fills. An external stylesheet styles the card, form, filter buttons, task rows and the completed (line-through) state using CSS variables, flexbox and grid.

Procedure:

1. Select the application to be built: a Task Tracker.

2. Create a folder Expt6 with the files index.html, css/style.css and js/app.js.

3. In index.html, add the form (id taskForm) with a text input and an Add button, the All / Open / Finished filter buttons, an empty <ul id="taskList"> and a <p id="counter">, and link css/style.css and js/app.js.

4. In css/style.css, define the colour variables and style the page, the card, the form, the buttons, the task rows, the completed state and the counter.

5. In js/app.js, select the page elements and load the saved tasks from localStorage into the tasks array; set the filter variable to "all".

6. Write createTask(text), which returns a task object with an id, the text and done set to false.

7. Write addTask(text), toggleTask(id) and deleteTask(id), which update the tasks array using push(), map() and filter(), and each end by calling saveAndRender().

8. Write saveAndRender(), which saves the tasks in localStorage and calls renderTasks().

9. Write the helper functions getVisible() (applies the current filter), countOpen() (counts tasks not done) and taskItem(t) (builds one list row with a checkbox, the text and a Delete button).

10. Write renderTasks(), which clears the list, appends a row for every visible task using forEach() and updates the counter.

11. Write handleSubmit(event) to prevent the page reload, trim the input, call addTask(text) when it is not empty and clear the input; attach it with form.addEventListener("submit", handleSubmit).

12. Write setFilter(name) and attach it to the click event of the filter buttons.

13. Call renderTasks() once at the end so that the saved tasks are shown on load.

14. Open index.html in a browser and test adding, completing, deleting and filtering tasks, and reload the page to confirm that the tasks are retained.

15. Push the code to GitHub and host it using GitHub Pages to obtain the deployment link.

Program

index.html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Task Tracker</title>
    <link rel="stylesheet" href="css/style.css" />
  </head>
  <body>
    <main class="page-shell">
      <section class="tracker-card">
        <h1>Task Tracker</h1>
        <p class="intro">Add a task, tick it off, or delete it.</p>

        <form id="taskForm">
          <input id="taskInput" type="text" placeholder="What needs doing?"
autocomplete="off" />
          <button type="submit">Add</button>
        </form>

        <div class="filters">
          <button type="button" class="filter active" data-filter="all">All</button>
          <button type="button" class="filter" data-filter="open">Open</button>
```

```
          <button type="button" class="filter" data-filter="done">Finished</button>
        </div>

        <ul id="taskList"></ul>
        <p id="counter" class="counter" aria-live="polite"></p>
      </section>
    </main>
    <script src="js/app.js"></script>
  </body>
</html>
```

css/style.css

```
:root {
  --ink: #314339;
  --heading: #244232;
  --accent: #c5a56a;
  --link: #496b55;
  --border: #d8ccb4;
  --surface: #fffdf8;
}

* { box-sizing: border-box; }

body {
  min-height: 100vh;
  margin: 0;
  padding: 30px 20px;
  color: var(--ink);
  font-family: Georgia, serif;
  background: #f4f0e7;
}

.page-shell { width: min(100%, 700px); margin: 0 auto; }

.tracker-card {
  padding: 28px;
  border: 1px solid var(--border);
  border-left: 5px solid var(--accent);
  border-radius: 10px;
  background: var(--surface);
  box-shadow: 0 4px 12px rgba(31, 77, 58, 0.08);
}

h1 { margin: 0; padding-bottom: 10px; border-bottom: 2px solid var(--border); color: var(--
heading); font-size: 2rem; }
.intro { margin: 16px 0 24px; line-height: 1.5; }

#taskForm { display: flex; gap: 10px; }
input {
  flex: 1;
  padding: 8px;
  border: 1px solid #b8a98a;
  border-radius: 4px;
  color: var(--ink);
  font: inherit;
  background: var(--surface);
}
input:focus { outline: 2px solid #e2c98e; border-color: var(--link); }

button {
  padding: 8px 18px;
  border: 1px solid var(--heading);
  border-radius: 4px;
```

```
  color: #fffaf0;
  background: var(--heading);
  cursor: pointer;
  font: 1rem Georgia, serif;
}
button:hover { background: var(--link); }

.filters { display: flex; gap: 8px; margin-top: 20px; }
.filter { padding: 6px 14px; color: var(--heading); background: #f8f5ef; border-color:
var(--border); font-size: 0.9rem; }
.filter:hover { background: #e7ede4; }
.filter.active { color: #fffaf0; background: var(--heading); border-color: var(--
heading); }

ul { margin: 18px 0 0; padding: 0; list-style: none; display: grid; gap: 10px; }
li { display: flex; align-items: center; gap: 12px; padding: 12px 14px; border: 1px solid
var(--border); border-radius: 4px; background: #f8f5ef; }
li span { flex: 1; line-height: 1.4; }
li.done span { color: #8a9a8f; text-decoration: line-through; }
li input[type="checkbox"] { flex: none; width: 18px; height: 18px; accent-color: var(--
heading); }
li .delete { margin: 0; padding: 4px 10px; color: var(--heading); background: transparent;
border-color: var(--border); font-size: 0.85rem; }
li .delete:hover { color: #fffaf0; background: #8a3b32; border-color: #8a3b32; }

.counter { margin: 18px 0 0; font-weight: bold; color: var(--heading); }
```

js/app.js

```
const form = document.getElementById("taskForm");
const input = document.getElementById("taskInput");
const list = document.getElementById("taskList");
const counter = document.getElementById("counter");
const filterButtons = document.querySelectorAll(".filter");

let tasks = JSON.parse(localStorage.getItem("tasks")) || [];
let filter = "all";

function createTask(text) {
  return {
    id: Date.now(),
    text: text,
    done: false
  };
}

function addTask(text) {
  tasks.push(createTask(text));
  saveAndRender();
}

function toggleTask(id) {
  tasks = tasks.map((t) => (t.id === id ? { ...t, done: !t.done } : t));
  saveAndRender();
}

function deleteTask(id) {
  tasks = tasks.filter((t) => t.id !== id);
  saveAndRender();
}

function setFilter(name) {
  filter = name;
  filterButtons.forEach((button) => {
```

```
    button.classList.toggle("active", button.dataset.filter === name);
  });
  renderTasks();
}

function saveAndRender() {
  localStorage.setItem("tasks", JSON.stringify(tasks));
  renderTasks();
}

function getVisible() {
  if (filter === "open") return tasks.filter((t) => !t.done);
  if (filter === "done") return tasks.filter((t) => t.done);
  return tasks;
}

function countOpen() {
  return tasks.filter((t) => !t.done).length;
}

function taskItem(t) {
  const item = document.createElement("li");
  item.className = t.done ? "done" : "";

  const checkbox = document.createElement("input");
  checkbox.type = "checkbox";
  checkbox.checked = t.done;
  checkbox.addEventListener("change", () => toggleTask(t.id));

  const label = document.createElement("span");
  label.textContent = t.text;

  const remove = document.createElement("button");
  remove.type = "button";
  remove.className = "delete";
  remove.textContent = "Delete";
  remove.addEventListener("click", () => deleteTask(t.id));

  item.append(checkbox, label, remove);
  return item;
}

function renderTasks() {
  list.innerHTML = "";
  getVisible().forEach((t) => {
    list.appendChild(taskItem(t));
  });
  counter.textContent = countOpen() + " left";
}

function handleSubmit(event) {
  event.preventDefault();
  const text = input.value.trim();
  if (text !== "") {
    addTask(text);
  }
  input.value = "";
}

form.addEventListener("submit", handleSubmit);
filterButtons.forEach((button) => {
  button.addEventListener("click", () => setFilter(button.dataset.filter));
});
```

```
renderTasks();
```

Output

Task Tracker (empty)

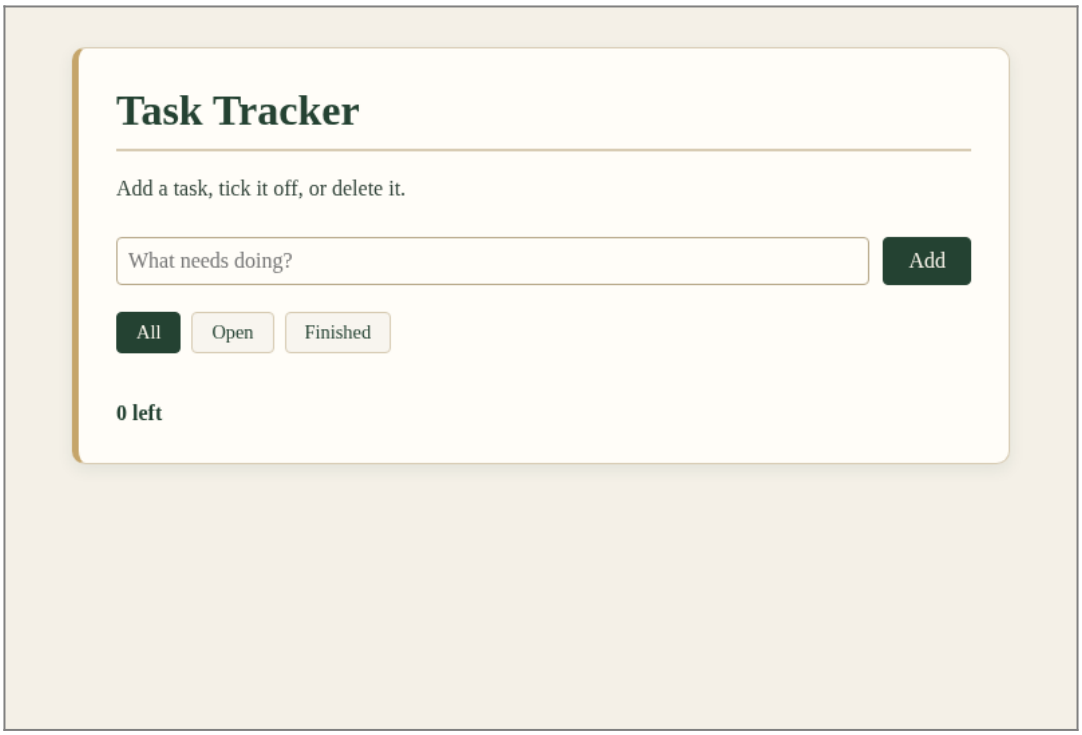


Figure 1: Task Tracker (empty state)

Task Tracker (with tasks)

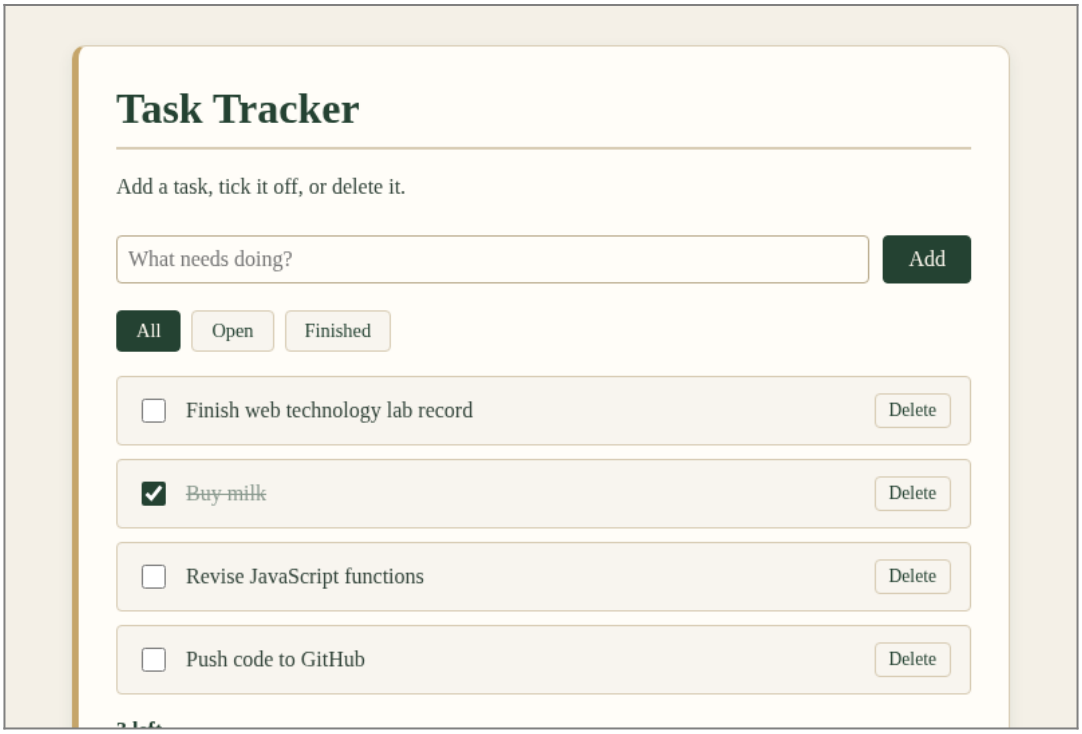


Figure 2: Task Tracker with tasks added, one of them completed (All filter)

Task Tracker (Open filter)

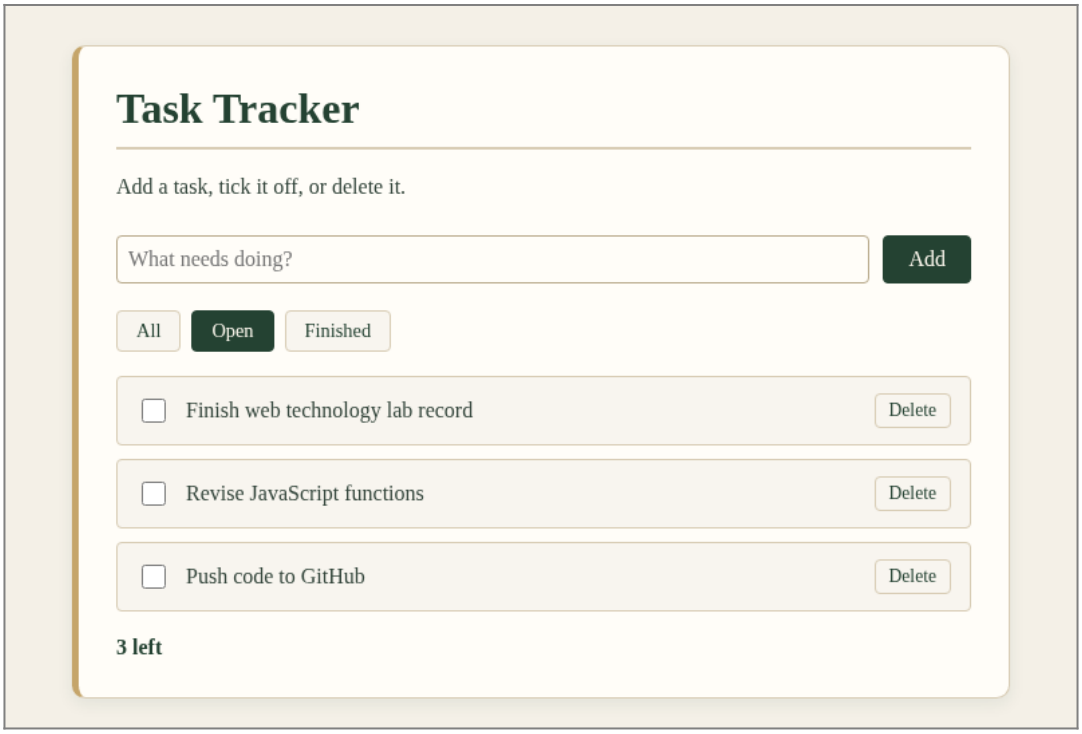


Figure 3: Only the open tasks shown after selecting the Open filter

Result

The Task Tracker was successfully designed and executed using JavaScript functions. Tasks can be added, completed, deleted and filtered, the list is redrawn by renderTasks() after every change, and the tasks are retained in localStorage after reloading. The output matched the expected behaviour.